Software Requirements Specification

for

Enterprise Concurrent Signal Broadcast Framework (ECSBF) Implementation

Version 0.1

Prepared by

Group – 04

Souvik	**Roy**
Krithiv	**Dharan SA**
Alisha	**Parveen**
Rittwika	**Datta**

Wipro private Ltd.

Table of Contents

Enterprise Concurrent Signal Broadcast Framework (ECSBF) Implementation

1. Introduction

The Enterprise Concurrent Signal Broadcast Framework (ECSBF) is a high-performance, distributed communication system designed to enable real-time, secure, and reliable signal propagation across a large network of interconnected nodes. The framework operates using a centralized Engine architecture responsible for orchestrating node connectivity, validating identities, maintaining persistent sessions, and broadcasting encrypted data signals concurrently to all active endpoints.

ECSBF is built to support enterprise-scale concurrency, where multiple distributed nodes may simultaneously send and receive signals without compromising system stability, security, or performance. The framework emphasizes fault tolerance, session resilience, and observability, ensuring uninterrupted operation even under transient network failures.

By integrating cryptographic protection, multi-threaded concurrency mechanisms, and a global identity registry, ECSBF provides a robust backbone for mission-critical distributed systems requiring synchronized communication, traceability, and controlled access.

1.1 Purpose

The purpose of this document is to define the software requirements specification (SRS) for the Enterprise Concurrent Signal Broadcast Framework (ECSBF). This document serves as a formal reference for the system's functional requirements, operational constraints, and quality attributes.

It describes how ECSBF establishes secure node-engine handshakes, verifies endpoint identities, manages persistent sessions, and performs unified signal broadcasting across distributed nodes. The SRS is intended to act as a baseline agreement between stakeholders and the development team, ensuring clarity, consistency, and traceability throughout the system lifecycle.

1.2 Document Conventions

This document follows standard Software Requirements Specification (SRS) formatting conventions:

- Section headings are presented in bold
- Requirement identifiers follow the format ECSBF_FR_XX
- Mandatory requirements are labeled as Must
- Recommended requirements are labeled as Should
- Optional capabilities are labeled as Optional
- Technical terminology is used consistently across sections for clarity and precision

All terminology related to nodes, engine, sessions, signals, and identity management is aligned with ECSBF architectural definitions.

1.3 Intended Audience and Reading suggestions

This document is intended for:

- Software architects
- Backend and systems engineers
- Security engineers
- QA and testing teams
- Project managers and technical stakeholders

Readers are expected to have a foundational understanding of:

- Distributed systems
- Client-server architectures
- Networking concepts
- Concurrent and multi-threaded systems
- Basic cryptographic principles

It is recommended that readers begin with Section 1 (Introduction) and Section 2 (Overall Description) to understand system context before reviewing detailed functional requirements.

1.4 Project scope:

The scope of ECSBF includes the design and specification of a centralized concurrency engine capable of managing secure communication between multiple distributed nodes. The system will:

- Provision and register nodes in a Global Identity Registry
- Authenticate and verify node identities
- Maintain persistent session states with automatic recovery
- Broadcast encrypted signals to all active nodes
- Support high-concurrency signal processing
- Provide multi-level diagnostic logging for observability

Out of scope items include UI-specific implementations and hardware-dependent optimizations unless explicitly stated.

1.5 References:

The following authoritative references have been used to support the architectural, security, and concurrency concepts defined in the Enterprise Concurrent Signal Broadcast Framework (ECSBF):

1. **Distributed Systems Concepts and Design**
   https://www.distributed-systems.net
   (Comprehensive academic and practical resource on distributed system architectures, coordination, and fault tolerance)

2. **NIST – National Institute of Standards and Technology (Cybersecurity & Cryptography)**
   https://www.nist.gov/cyberframework
   (Industry-standard guidelines for secure communication, encryption, identity verification, and system resilience)

3. **Cloudflare Learning Center – Networking & Security**
   https://www.cloudflare.com/learning/
   (Clear explanations of network protocols, encryption, distributed systems, and secure data transmission)

4. **IBM Documentation – Distributed Systems & Event-Driven Architecture**
   https://www.ibm.com/docs
   (Enterprise-grade references for concurrent processing, messaging systems, and system observability)

5. **Apache Kafka Documentation (Event Broadcasting & Concurrency Models)**
   https://kafka.apache.org/documentation/
   (Widely adopted industry reference for high-throughput, fault-tolerant signal broadcasting systems)

2. Overall Description

This section provides a high-level overview of the Enterprise Concurrent Signal Broadcast Framework (ECSBF), outlining its architectural context, core capabilities, operating conditions, constraints, documentation approach, and foundational assumptions.

2.1 Product Perspective

The Enterprise Concurrent Signal Broadcast Framework (ECSBF) is designed as a centralized-engine–driven distributed communication system. The system consists of a Core Engine and multiple distributed endpoint nodes that interact through secure, persistent connections.

The Engine acts as the authoritative control unit responsible for:

- Node provisioning and identity management
- Secure handshake initiation
- Authentication and session validation
- Concurrent signal processing
- Unified broadcast of signals to all active nodes

Each endpoint node functions as an independent signal producer and consumer, capable of sending data packets to the Engine and receiving broadcasted signals from other nodes. ECSBF is architected to operate reliably in enterprise-scale environments where high concurrency, fault tolerance, and secure communication are mandatory.

2.2 Product Features

The Enterprise Concurrent Signal Broadcast Framework (ECSBF) provides the following key features to support secure, scalable, and real-time distributed communication:

1. **Secure Node–Engine Handshake**
   ECSBF establishes a secure handshake protocol between each endpoint node and the central Engine to initiate trusted communication and signal synchronization.

2. **Global Identity Registry**
   The system maintains a persistent registry that stores unique identities and profiles of all provisioned nodes, ensuring controlled access to framework services.

3. **Node Provisioning Mechanism**
   New nodes must be formally provisioned and registered before participating in signal exchange, preventing unauthorized or rogue node access.

4. **Mandatory Identity Verification**
   Every session request undergoes strict authentication and verification to ensure that only legitimate nodes can establish or restore sessions.

5. **Persistent Session Management**
   ECSBF maintains active session states for verified nodes and supports automatic session restoration in case of temporary network disruptions.

6. **High-Concurrency Multi-Threaded Engine**
   The central Engine is designed to process multiple concurrent signal requests efficiently, enabling enterprise-scale parallel communication.

7. **Unified Signal Broadcasting**
   Any signal or data packet transmitted by a node is broadcasted by the Engine to all currently active nodes in the network.

8. **Source Identification and Traceability**
   Each broadcasted signal is appended with a source identifier, allowing nodes to identify the origin of the signal for tracking and auditing purposes.

9. **Cryptographic Protection of Data**
   ECSBF integrates encryption mechanisms to protect sensitive identifiers, credentials, and signal payloads during transmission.

10. **Secure Payload Decryption**
    Endpoint nodes are equipped with decryption modules to securely decode received payloads while preserving data integrity.

11. **Network Identity Resolution**
    The system optionally supports resolution of network-level identifiers (such as IP addresses) into human-readable node identities.

12. **Multi-Level Diagnostic Logging**
    ECSBF provides a comprehensive logging framework supporting FATAL, INFO, WARNING, and DEBUG levels to enhance observability, debugging, and system monitoring.

2.3 Operating Environment

The ECSBF framework is intended to operate in modern distributed computing environments and supports deployment across heterogeneous systems.

Typical operating environment includes:

- **Operating Systems**: Linux (preferred), Windows (supported)
- **Execution Model**: Multi-threaded server-side engine with persistent socket-based connections
- **Network**: TCP/IP-based enterprise or cloud networks
- **Deployment Models**: On-premise, cloud-based, or hybrid environments

The system is designed to remain operational under varying network conditions, including intermittent connectivity and fluctuating latency.

2.4 Design and Implementation Constraints

The following constraints apply to the ECSBF system:

- All nodes must be provisioned in the Global Identity Registry prior to access
- Only authenticated and verified nodes may establish active sessions
- The Engine must support concurrent processing without race conditions or deadlocks
- Encryption mechanisms must comply with industry-accepted cryptographic standards
- Session state data must be efficiently managed to avoid memory leaks or resource exhaustion
- Logging mechanisms must not significantly degrade runtime performance

These constraints ensure security, scalability, and reliability in enterprise environments.

2.5 User Documentation

User and developer documentation for ECSBF will include:

- System architecture overview
- Node onboarding and provisioning guides
- Configuration and deployment manuals
- Security and cryptographic guidelines
- Diagnostic logging and troubleshooting references

All documentation will be maintained in a version-controlled repository and updated alongside system releases to ensure accuracy and consistency.

2.6 Assumptions and Dependencies

The ECSBF system is developed under the following assumptions and dependencies:

- Nodes have stable network connectivity to the central Engine
- All participating systems support required cryptographic libraries
- Time synchronization across nodes is reasonably accurate
- The underlying operating system supports multi-threaded execution

- Network infrastructure permits persistent socket connections
- Logging storage is sufficient for diagnostic data retention

Any deviation from these assumptions may impact system performance or reliability.

3. System Features

This section describes the major system features of the Enterprise Concurrent Signal Broadcast Framework (ECSBF). Each feature is presented with a description, priority, operational behavior, and traceable functional requirements.

3.1 Secure Node Registration and Session Establishment

3.1.1 Description and Priority

Priority: High

The ECSBF system shall allow endpoint nodes to establish a secure and authenticated connection with the central Engine. Only nodes that are formally provisioned in the Global Identity Registry are permitted to initiate sessions. This feature ensures controlled access, identity validation, and protection against unauthorized signal injection.

3.1.2 Stimulus / Response Sequences

- **Stimulus:** A node attempts to connect to the Engine
- **Response:**
    - Engine initiates a secure handshake
    - Node identity is verified against the Global Identity Registry
    - If verified, a persistent session is established
    - If verification fails, the connection is rejected and logged

3.1.3 Functional Requirements

- **ECSBF_FR_01: The system shall establish a secure handshake protocol between the node and the Engine.**

  The ECSBF system shall allow a node to initiate a connection request with the central Engine. The Engine validates the node identity using the Global Identity Registry. If the identity is valid, a secure session is established; otherwise, the connection is rejected and logged.

- **ECSBF_FR_02: All nodes must be provisioned in the Global Identity Registry prior to access.**

- All endpoint nodes must be formally provisioned within the Global Identity Registry prior to accessing ECSBF services. Provisioning ensures that only authorized and registered nodes can participate in signal transmission and reception.

- **ECSBF_FR_03: All session requests must undergo mandatory identity verification.**

  Every session request from a node shall undergo mandatory identity authentication. This verification prevents unauthorized access and protects the system from malicious or untrusted signal injection.

- **ECSBF_FR_04: The Engine shall maintain persistent session states for verified nodes.**

  The Engine shall maintain active sessions for all verified nodes. These sessions preserve communication state and support automatic restoration during temporary network disruptions.

3.2 Concurrent Signal Processing and Broadcasting

3.2.1 Description and Priority

Priority: High

The ECSBF Engine shall support high-performance concurrent processing of signals originating from multiple distributed nodes. Any signal received from a node shall be reliably broadcasted to all currently active nodes in the system.

3.2.2 Stimulus / Response Sequences

- **Stimulus:** A node transmits a signal payload to the Engine
- **Response:**
  - Engine validates the active session
  - Signal is tagged with the source node identifier
  - Signal is broadcasted to all active nodes concurrently

3.2.3 Functional Requirements

- **ECSBF_FR_05: The Engine shall process simultaneous signal requests using a multi-threaded concurrency model.**

  The ECSBF system must utilize a high-performance concurrent processing engine to handle multiple signal requests simultaneously. This ensures scalability and uninterrupted operation under high node traffic.

- **ECSBF_FR_06: Any signal dispatched by a node must be broadcasted to all active nodes with source traceability.**

Any signal or data packet sent by a node shall be broadcasted to all currently active nodes. Each signal shall include a source identifier to enable traceability and auditing.

3.3 Global Identity and Network Resolution Management

3.3.1 Description and Priority

Priority: Medium

The ECSBF system maintains a centralized identity registry that maps technical identifiers (such as IP addresses or session tokens) to human-readable node identities. This enhances traceability, monitoring, and system administration.

3.3.2 Stimulus / Response Sequences

- **Stimulus:** Engine or administrator requests identity resolution
- **Response:**
    - System resolves network-level identifiers to node profiles
    - Human-readable identity information is returned

3.3.3 Functional Requirements

- **ECSBF_FR_07: The system shall maintain a persistent Global Identity Registry.**

  A persistent Global Identity Registry shall be maintained to store and manage identity profiles of all provisioned nodes. The registry serves as the authoritative source for authentication and monitoring.

- **ECSBF_FR_09: The system should provide logical-to-human-readable identity resolution.**

  The system may provide a mechanism to resolve network-level identifiers such as IP addresses into human-readable node identities. This feature assists in system monitoring and diagnostics.

3.4 Secure Signal Transmission and Payload Handling

3.4.1 Description and Priority

Priority: Medium

ECSBF shall ensure that all sensitive data—including identifiers, credentials, and signal payloads—are protected during transmission using cryptographic techniques. Endpoint nodes shall securely decrypt received payloads while maintaining data integrity.

3.4.2 Stimulus / Response Sequences

- **Stimulus:** A node sends or receives an encrypted signal

- **Response:**
    - Payload is encrypted before transmission
    - Receiving node decrypts and validates payload integrity

3.4.3 Functional Requirements

- **ECSBF_FR_10: The system should encrypt sensitive identifiers and payload data.**

  Encryption mechanisms should be applied to protect sensitive identifiers, credentials, and signal payloads during transmission. This enhances confidentiality and data security across the network.

- **ECSBF_FR_11: The system should provide secure decryption mechanisms at endpoints.**

  The framework should provide secure decryption mechanisms at the endpoint nodes. Decryption ensures that received signal payloads are correctly interpreted while preserving data integrity.

3.5 Session Termination and Resource Management

3.5.1 Description and Priority

Priority: Medium

The ECSBF Engine shall gracefully terminate sessions when nodes disconnect, ensuring that all session-related resources are released to maintain system efficiency and stability.

3.5.2 Stimulus / Response Sequences

- **Stimulus:** A node disconnects or session timeout occurs
- **Response:**
    - Engine terminates the session
    - Session data is purged
    - Resources are released

3.5.3 Functional Requirements

- **ECSBF_FR_08:** The system shall formally terminate and purge session data upon node disconnection.

  The Engine shall formally terminate sessions when a node disconnects or becomes inactive. All associated session data shall be purged to ensure efficient resource utilization.

3.6 Diagnostic Logging and Observability

3.6.1 Description and Priority

Priority: High

ECSBF shall provide a comprehensive logging mechanism to support operational monitoring, debugging, and incident analysis through multiple verbosity levels.

3.6.2 Stimulus / Response Sequences

- **Stimulus:** System events, errors, or state transitions occur
- **Response:**
    - Logs are recorded at appropriate severity levels
    - Logs are made available for diagnostics and auditing

3.6.3 Functional Requirements

- **ECSBF_FR_12: The system must support diagnostic logging with FATAL, INFO, WARNING, and DEBUG levels.**

  The system must implement a diagnostic logging mechanism supporting FATAL, INFO, WARNING, and DEBUG levels. These logs aid in monitoring, debugging, and system analysis.

4. External Interface Requirements

This section describes the external interfaces through which the Enterprise Concurrent Signal Broadcast Framework (ECSBF) interacts with users, hardware, software components, and communication networks.

4.1 User Interfaces

ECSBF is primarily a backend, engine-driven framework and does not mandate a graphical user interface. Interaction with the system is performed through:

Command-line interfaces (CLI) for engine startup, configuration, and monitoring

Configuration files for node provisioning and security settings

Log outputs for operational visibility and diagnostics

Administrative users may use terminal-based tools or dashboards built on top of ECSBF to observe node connectivity, session states, and broadcast activity. Any future graphical interfaces are considered optional extensions and are outside the core scope of ECSBF.

4.2 Hardware Interfaces

ECSBF does not require specialized hardware and operates on standard computing infrastructure. The framework interfaces with hardware components only through the underlying operating system.

Minimum recommended hardware requirements include:

Engine Node:

- Multi-core processor
- Minimum 2 GB RAM (scalable based on node count)
- Persistent storage for identity registry and logs
- Network interface supporting TCP/IP communication

Endpoint Nodes:

Standard processor

Minimum 1 GB RAM

Network interface for Engine connectivity

The system remains hardware-agnostic and scalable across physical, virtual, and cloud-based environments.

4.3 Software Interfaces

The Enterprise Concurrent Signal Broadcast Framework (ECSBF) interfaces with multiple software components to support secure communication, concurrency management, identity handling, and observability. These interfaces are designed to be modular, extensible, and platform-agnostic.

Operating System Interface

ECSBF operates on modern operating systems that support multi-threading, networking, and secure process execution.

- **Primary Support:** Linux-based operating systems
- **Secondary Support:** Windows-based operating systems
  The framework relies on OS-level services for process scheduling, thread management, network sockets, and file system access.

Networking Stack Interface

ECSBF interfaces directly with the system's networking stack to establish and manage persistent connections between the Engine and endpoint nodes.

- TCP/IP socket interfaces for reliable communication
- Support for concurrent connection handling
- OS-managed network buffers and I/O multiplexing mechanisms

Cryptographic Services Interface

The system integrates with standard cryptographic libraries to provide:

- Encryption of sensitive identifiers and signal payloads
- Secure key handling and validation
- Decryption and integrity verification at endpoint nodes

All cryptographic operations are abstracted to ensure future compatibility with updated security standards.

Identity and Configuration Management Interface

ECSBF interfaces with configuration services or files to manage:

- Node provisioning data
- Global Identity Registry persistence
- Security credentials and access policies
- Engine runtime configuration parameters

These interfaces allow administrators to modify system behavior without recompiling core components.

Logging and Monitoring Interface

The framework integrates with logging subsystems to record operational and diagnostic information.

- Support for multi-level logs (FATAL, INFO, WARNING, DEBUG)
- Structured log outputs for analysis and auditing
- Compatibility with external log aggregation and monitoring tools

Integration and Extension Interfaces

ECSBF is designed to interoperate with external systems such as:

- Monitoring and observability platforms
- Deployment and orchestration tools
- Enterprise management dashboards

Standardized interfaces ensure ECSBF can be extended or embedded within larger distributed ecosystems.

4.4 Communications Interfaces

ECSBF relies on standard network communication mechanisms to enable secure and reliable data exchange between the Engine and endpoint nodes.

- Network Protocols: TCP/IP-based communication
- Session Model: Persistent, stateful connections
- Security Layer: Encrypted communication channels for sensitive data
- Data Format: Structured signal packets with embedded source identifiers

The communication interface is designed to support high concurrency, low latency, and resilience against transient network failures.

5. Other Nonfunctional Requirements

This section defines the non-functional requirements that govern the performance, safety, reliability, and overall quality of the Enterprise Concurrent Signal Broadcast Framework (ECSBF).

5.1 Performance Requirements

The ECSBF system shall be designed to operate efficiently under high concurrency and real-time communication demands.

- The Engine shall support simultaneous connections from multiple distributed nodes without performance degradation.
- Signal broadcasting shall occur with minimal latency, ensuring near real-time propagation across active nodes.
- The system shall scale horizontally to accommodate increasing node counts and signal volumes.
- Session restoration following transient network failures shall occur without manual intervention and with minimal delay.
- Logging and monitoring operations shall not significantly impact core signal processing performance.

5.2 Safety and Security Requirements

ECSBF shall ensure safe and secure operation to prevent unauthorized access, data leakage, and system instability.

- Only provisioned and authenticated nodes shall be allowed to participate in signal exchange.
- Sensitive identifiers, credentials, and payload data shall be protected using cryptographic mechanisms during transmission.
- Session termination shall be handled gracefully to prevent resource leakage or orphaned sessions.
- The system shall be resilient to partial network failures and recover without compromising data integrity.
- Diagnostic logs shall not expose sensitive information and shall follow secure access controls.

5.3 Software Quality Attributes

The ECSBF framework shall adhere to the following software quality attributes to ensure long-term maintainability and operational excellence:

- **Reliability**: The system shall maintain stable operation under continuous load and recover gracefully from failures.
- **Scalability**: The architecture shall support growth in node count, signal frequency, and deployment size.
- **Maintainability**: Modular design and clear separation of concerns shall allow efficient updates and enhancements.
- **Observability**: Multi-level logging (FATAL, INFO, WARNING, DEBUG) shall provide clear insight into system behavior.
- **Portability**: The framework shall be deployable across Linux and Windows environments with minimal configuration changes.
- **Security**: Strong identity verification, encrypted communication, and controlled session management shall be enforced.

6. Other Requirements

This section specifies additional requirements and considerations that do not fall strictly under functional or non-functional categories but are essential for the successful deployment, operation, and evolution of the Enterprise Concurrent Signal Broadcast Framework (ECSBF).

6.1 Regulatory and Compliance Considerations

ECSBF shall be designed in alignment with commonly accepted enterprise security and data protection practices. Where applicable, the system should support compliance with organizational policies and regulatory standards related to secure communication, data handling, and access control.

6.2 Deployment and Configuration Requirements

- The ECSBF Engine and endpoint nodes shall support environment-based configuration to allow flexible deployment across development, testing, and production environments.
- Configuration parameters related to security, concurrency limits, logging levels, and session handling shall be externally configurable.
- The system shall support automated deployment mechanisms where applicable.

6.3 Monitoring and Operational Support

- ECSBF shall provide sufficient diagnostic information to support system monitoring, auditing, and troubleshooting.
- Operational metrics such as active sessions, node connectivity, and signal throughput should be observable through logs or external monitoring integrations.
- The framework shall support graceful startup, shutdown, and restart procedures.

6.4 Extensibility and Future Enhancements

- The ECSBF architecture shall be modular to allow future enhancements without major refactoring.
- Support for additional communication protocols, security algorithms, or observability tools may be incorporated in future releases.
- Backward compatibility should be maintained wherever feasible to protect existing deployments.

6.5 Backup and Recovery Considerations

- Persistent data such as identity registries and configuration artifacts should be backed up regularly.
- The system should support recovery mechanisms to restore operational state following failures or system restarts.
- Session recovery mechanisms shall prioritize system consistency and data integrity.

6.6 Documentation and Training

- Technical documentation shall be maintained to reflect system architecture, configuration, and operational procedures.
- Deployment guides and troubleshooting references shall be provided to support administrators and operators.
- Documentation updates shall accompany significant system changes or enhancements.

Appendix A: Glossary

ECSBF
Enterprise Concurrent Signal Broadcast Framework – A distributed, high-concurrency communication framework designed for secure, real-time signal propagation across multiple networked nodes.

Engine
The centralized core component of ECSBF responsible for node authentication, session management, concurrent signal processing, and unified signal broadcasting.

Node
An endpoint system or process that connects to the ECSBF Engine to send and receive signals. Nodes may act as signal producers, consumers, or both.

Endpoint Node
A provisioned ECSBF node that actively participates in communication through a verified session with the Engine.

Global Identity Registry
A persistent repository that stores unique identities, credentials, and metadata of all provisioned ECSBF nodes.

Node Provisioning
The formal process of registering a node into the Global Identity Registry before it is allowed to access ECSBF services.

Handshake
A secure initialization process performed between a node and the Engine to establish trust and initiate a session.

Identity Verification
The authentication process that validates a node's identity against the Global Identity Registry before granting access.

Session
A persistent logical connection between a node and the Engine that maintains state information for reliable communication.

Persistent Session Management
The capability of the Engine to maintain and restore active sessions during transient network interruptions.

Signal
A structured data packet transmitted by a node and broadcasted by the Engine to other active nodes.

Signal Broadcast
The process by which a signal originating from one node is propagated to all active nodes in the ECSBF network.

Source Identifier
A unique identifier appended to each signal to indicate the originating node for traceability and auditing.

Concurrency Engine
The multi-threaded processing mechanism within the ECSBF Engine that enables simultaneous handling of multiple node connections and signals.

Multi-Threading
A programming model that allows multiple threads of execution to run concurrently within the Engine.

Cryptographic Protection
The use of encryption techniques to secure sensitive data such as credentials, identifiers, and signal payloads during transmission.

Payload
The actual data content carried within a signal packet.

Payload Decryption
The process performed at the receiving node to decode encrypted signal payloads and restore original data.
Network Identity Resolution
The mechanism used to map low-level network identifiers (e.g., IP addresses) to human-readable node identities.

Observability
The ability to monitor internal system states through logs, metrics, and diagnostics.

Diagnostic Logging
The recording of system events and operational data for monitoring, debugging, and auditing purposes.

FATAL Log Level
Logs indicating critical errors that may cause system shutdown or severe malfunction.

INFO Log Level
Logs providing general operational information about normal system behavior.

WARNING Log Level
Logs indicating abnormal conditions that do not immediately disrupt system operation but may require attention.

DEBUG Log Level
Logs providing detailed technical information intended for troubleshooting and analysis.

Session Termination
The controlled shutdown and cleanup of a node's session when a node disconnects or times out.

Fault Tolerance
The system's ability to continue operating correctly despite partial failures or network disruptions.

Scalability
The capability of ECSBF to handle increasing numbers of nodes and signal traffic without performance degradation.

SRS
Software Requirements Specification – A formal document that defines the functional and non-functional requirements of the ECSBF system.